STAT 508 - HW6

Ana Camargo

2023-05-05

Chapter 9

For this homework, the 2 problems I selected were **problem 5 and problem 7.**

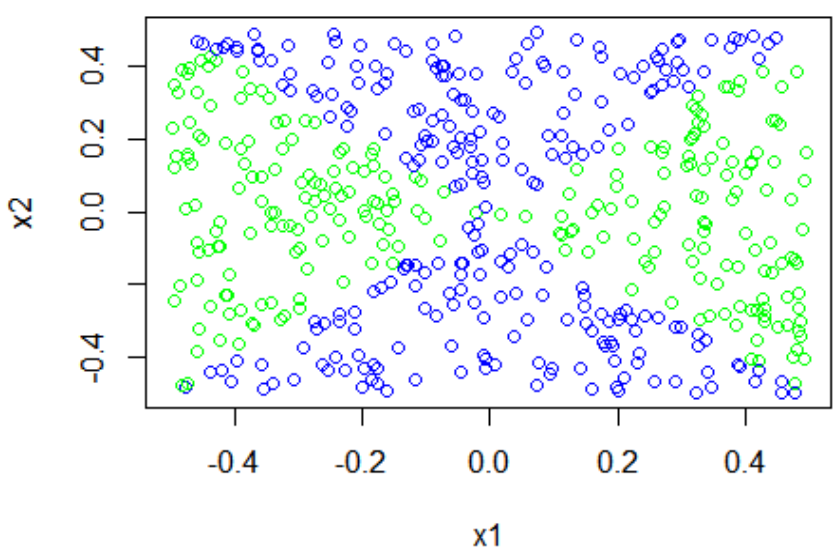
Problem 5

a) Generate a data set with n = 500 and p = 2, such that the observations belong to two classes with a quadratic decision boundary between them.

```
set.seed(5)
x1 <- runif(500) - 0.5
x2 <- runif(500) - 0.5
y <- 1*(x1^2 - x2^2 > 0)
```

b) Plot the observations, colored according to their class labels. Your plot should display X1 on the x-axis, and X2 on the yaxis.

```
plot(x1,x2,col=ifelse(y,'green','blue'))
```

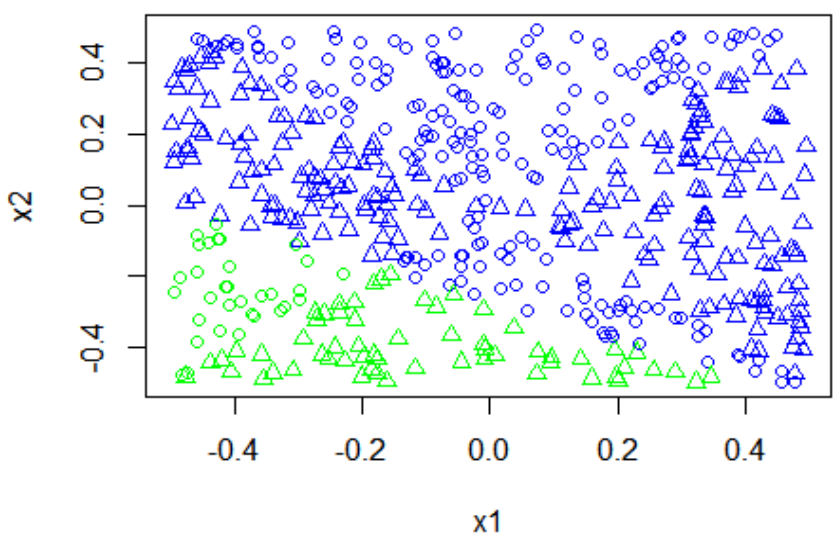


c) Fit a logistic regression model to the data, using X1 and X2 as predictors.

```
glm.fit = glm(y~. ,family='binomial', data=data.frame(x1,x2,y))
glm.fit

##
## Call:  glm(formula = y ~ ., family = "binomial", data = data.frame(x1,
##     x2, y))
##
## Coefficients:
## (Intercept)           x1           x2
##    -0.03150     -0.06176     -0.11509
##
## Degrees of Freedom: 499 Total (i.e. Null);  497 Residual
## Null Deviance:       693
## Residual Deviance: 692.9     AIC: 698.9
```

d) Apply this model to the training data in order to obtain a predicted class label for each training observation. Plot the observations, colored according to the predicted class labels. The decision boundary should be linear.

```
glm.pred = predict(glm.fit,data.frame(x1,x2))   # prediction using glm.fit mo
del
plot(x1,x2,col=ifelse(glm.pred>0,'green','blue'),pch=ifelse(as.integer(glm.pr
ed>0) == y,1,2))
```



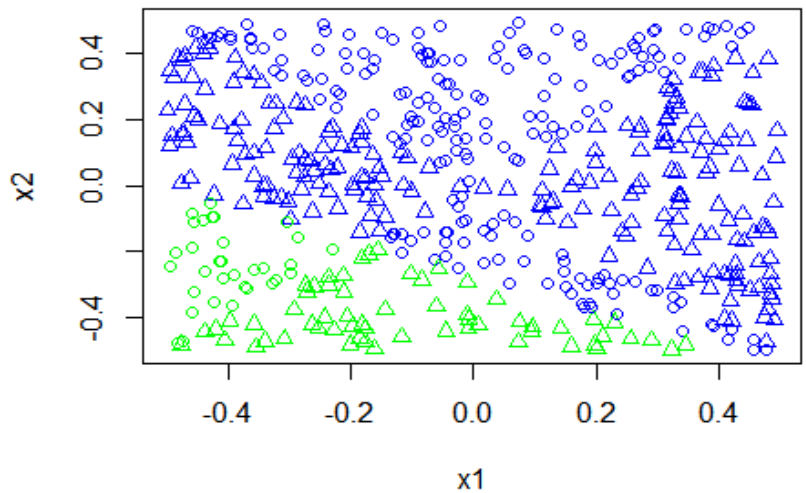
The predicted class is shown in colors green and blue. Points that are correctly classified are shown as a circle, while triangles are misclassified points. We can see that there are a significant number of misclassified points.

e) Now fit a logistic regression model to the data using non-linear functions of X1 and X2 as predictors (e.g. X1^2, X1*X2, log(X2), and so forth).

```
glm.fit2 = glm(y ~ poly(x1,2) + log(x2) + (x1*x2) , family='binomial', data=d
ata.frame(x1,x2,y))

glm.fit2

##
## Call:  glm(formula = y ~ poly(x1, 2) + log(x2) + (x1 * x2), family = "bino
mial",
##     data = data.frame(x1, x2, y))
##
## Coefficients:
##  (Intercept)  poly(x1, 2)1  poly(x1, 2)2       log(x2)            x1
##       26.551       -35.777       253.755         2.534            NA
##           x2         x1:x2
##      -88.255        17.742
##
## Degrees of Freedom: 257 Total (i.e. Null);  252 Residual
##   (242 observations deleted due to missingness)
## Null Deviance:       357.5
## Residual Deviance: 57.36     AIC: 69.36
```

f) Apply this model to the training data in order to obtain a predicted class label for each training observation. Plot the observations,colored according to the predicted class labels. The decision boundary should be obviously non-linear.

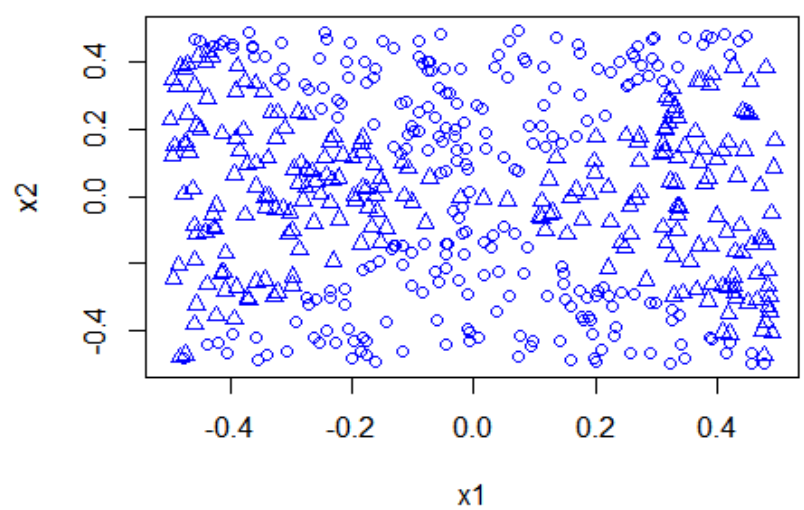
```
glm.pred2 = predict(glm.fit2,data.frame(x1,x2))     # prediction using glm.fi
t2 model

## Warning in log(x2): NaNs produced

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type
== :
## prediction from a rank-deficient fit may be misleading

plot(x1,x2, col=ifelse(glm.pred>0,'green','blue'), pch=ifelse(as.integer(glm.
pred>0) == y,1,2))
```



In this case we have predicted class and correct classification all mixed in the plot, so it is hard to say that this model is doing a good job as a classifier.

g) Fit a support vector classifier to the data with X1 and X2 as predictors. Obtain a class prediction for each training observation. Plot the observations, colored according to the predicted class labels.

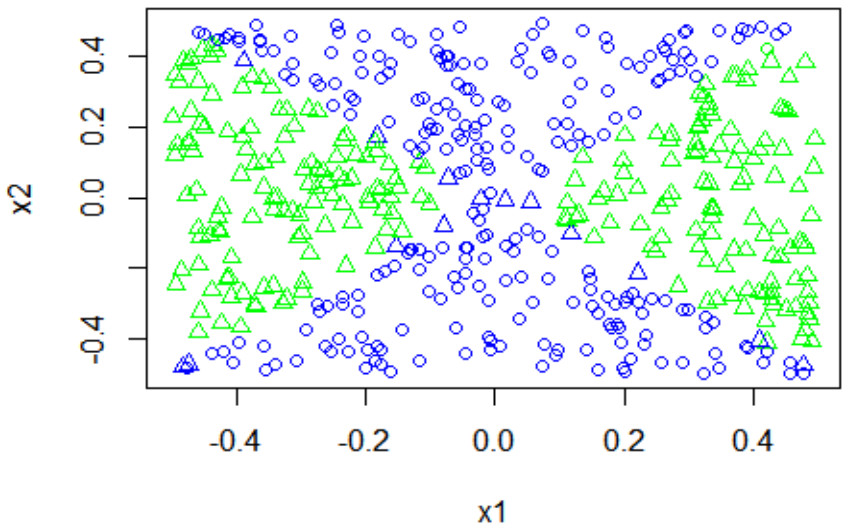
```
library(e1071)
svm.fit = svm(y~., data=data.frame(x1,x2,y=as.factor(y)), kernel='linear')
svm.pred = predict(svm.fit, data.frame(x1,x2),  type='response')
plot(x1,x2, col=ifelse(svm.pred == 0,'blue','green'), pch=ifelse(svm.pred ==
y,1,2))
```



Using a linear kernel we can see that it does not do a good job as a classifier. All the points were classified in one class (blue color) while it clearly should have been two classes as we can see from points and triangles.

h) Fit a SVM using a non-linear kernel to the data. Obtain a class prediction for each training observation. Plot the observations, colored according to the predicted class labels.

```
svm.fit2 = svm(y~., data=data.frame(x1,x2,y=as.factor(y)), kernel='polynomial
', degree=2)
svm.pred2 = predict(svm.fit2, data.frame(x1,x2), type='response')
plot(x1,x2, col=ifelse(svm.pred2 == 0 ,'blue','green'), pch=ifelse(svm.pred =
= y,1,2))
```



The non-linear kernel svm model is doing a good classification, overall. We can see how this plot is similar to the first one where we can see the two colors for each true class. Also, we can see that there are just a few misclassified points (triangles in the blue region).

i) Comment on your results.

```
table(y,as.integer(glm.pred>0))

##
## y     0   1
##   0 194  60
##   1 214  32
```

In this first model (log. reg.) from the confusion matrix we can see that accuracy is just (214+60)/500 = 0.548 which is low.

```
table(y,as.integer(glm.pred2>0))# confusion matrix

##
## y     0   1
##   0 121  11
##   1   7 119
```

The log. reg. with non-linear predictors have an accuracy even lower, this performs the worst.

```
table(y,svm.pred) # confusion matrix

##    svm.pred
## y     0   1
##   0 254   0
##   1 246   0

mean(y == svm.pred) # accuracy

## [1] 0.508
```

For the SVM with a linear kernel, the overall accuracy is of 0.508 which is still low.

```
table(y,svm.pred2) # confusion matrix

##    svm.pred2
## y     0   1
##   0 253   1
##   1  14 232

mean(y == svm.pred2) # accuracy

## [1] 0.97
```

The non-linear kernel SVM model is the best model out of the four, as it has an accuracy of 0.97.

Problem 7

a) Create a binary variable that takes on a 1 for cars with gas mileage above the median, and a 0 for cars with gas mileage below the median.

```
library(ISLR2)
data(Auto)
Auto$mpg=ifelse(Auto$mpg>median(Auto$mpg),1,0)
table(Auto$mpg)

##
##   0   1
## 196 196
```

b) Fit a support vector classifier to the data with various values of cost, in order to predict whether a car gets high or low gas mileage. Report the cross-validation errors associated with different values of this parameter. Comment on your results.

```
library(e1071)
costs = data.frame(cost=seq(0.1,200,length.out = 15))
svm.tune = tune(svm , mpg~., data=Auto, ranges=costs, kernel='linear')
svm.tune

##
## Parameter tuning of 'svm':
##
```

```
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost
##   0.1
##
## - best performance: 0.1002298
```

We can see that the best performance is gotten with a cost value of 0.10. The first plot in section (d) shows this result as the lowest error.

c) Now repeat (b), this time using SVMs with radial and polynomial basis kernels, with different values of gamma and degree and cost. Comment on your results.

```
parameters = data.frame(cost=seq(0.10,200,length.out = 5),degree=seq(1,200,le
ngth.out = 5))
svm.poly = tune(svm,mpg~., data=Auto, ranges=parameters, kernel='polynomial')
svm.poly
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##     cost degree
##  150.025      1
##
## - best performance: 0.1002982
```

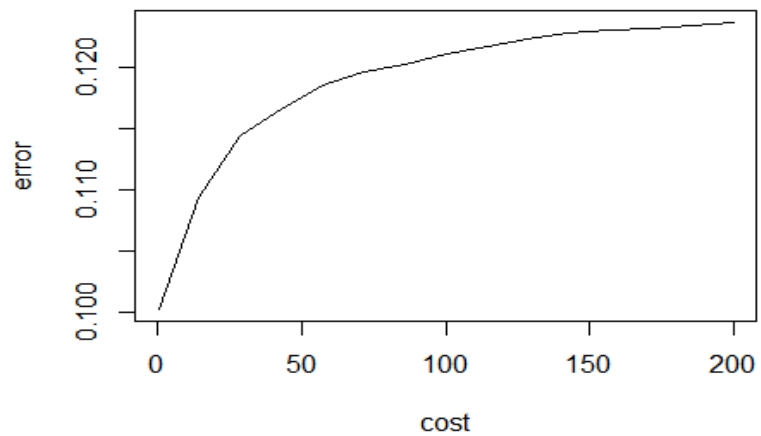
```
parameters2 = data.frame(cost=seq(0.10,200,length.out = 5),gamma=seq(0.1,200,
length.out = 5))
svm.radial = tune(svm,mpg~., data=Auto, ranges=parameters2, kernel='radial')
svm.radial
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##   0.1   0.1
##
## - best performance: 0.07309855
```

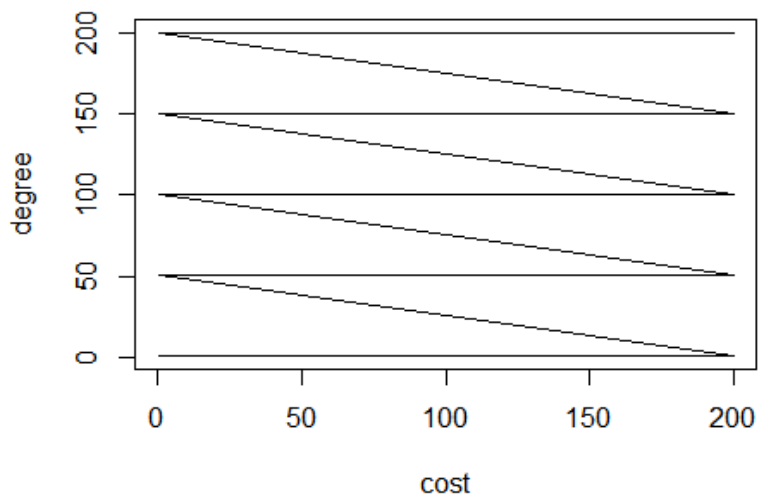
From above, we can see that when using a polynomial kernel and varying gamma and degree, the best performance is with a cost of 150 and degree of 1. When using a radial kernel, the best parameters are 0.1 for the cost and gamma 0.1. The 2nd and 3rd plots in part (d) show these results.

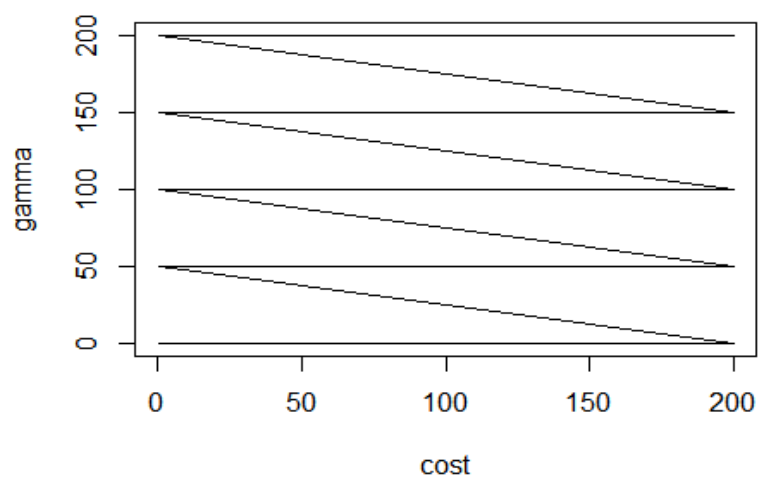
d) Make some plots to back up your assertions in (b) and (c).

```
plot(svm.tune$performance[,c(1,2)],type='l')
```




```
plot(svm.poly$performance[,c(1,2)],type='l')
```




```
plot(svm.radial$performance[,c(1,2)],type='l')
```